

Hockey Prediction

Author: Louis-Pierre Gagnaux

Public Git Repo: https://github.com/gagnauxl/hockey_prediction

Daten Swiss National League: <https://www.sihf.ch/de/game-center/national-league/#/standing/rank/asc/page/0/>

```
In [1]: import data_service as dp
        from team import Team
        import ranking_regression as rr
        import lin_regression as lr
        import knn_regression as knn
        import elo_regression as elo
        import rpart_regression as rp
        import random_forest as rf
```

```
In [2]: data = dp.load_data()
        data_proj=data[['Datum', 'Home', 'Away', 'Resultat']].tail(5)
        print(f>Last 5 games:\n{data_proj.to_string()}")
```

Last 5 games:

	Datum	Home	Away	Resultat
359	09/03/2026	Genève-Servette HC	EHC Kloten	07:00
360	09/03/2026	HC Lugano	EV Zug	04:01
361	09/03/2026	Lausanne HC	HC Ajoie	02:03
362	09/03/2026	SC Rapperswil-Jona Lakers	HC Ambri-Piotta	06:02
363	09/03/2026	SCL Tigers	ZSC Lions	00:02

```
In [3]: X, y, df = dp.load()           # Lädt die Daten, bereitet sie vor und teilt sie in Features (X) und Zielvariable (y) auf
        print(f>Last 10 values of df: \n{df[['Home_Id', 'Away_Id', 'Home', 'Away', 'Resultat', 'OT/SO', 'Points']].head(10).to_string()}")
        print(f>Features: {X.shape}")
        #X.info()
        print(X.head())
        print(f">\nShape of target variable y: {y.shape}\n{y[:5]}")
```

Last 10 values of df:

	Home_Id	Away_Id	Home	Away	Resultat	OT/SO	Points
0	0	10	HC Ambri-Piotta	EHC Kloten	02:01	NaN	3
1	1	12	HC Davos	Lausanne HC	04:01	NaN	3
2	2	11	Fribourg-Gottéron	HC Lugano	03:02	OT	2
3	3	8	Genève-Servette HC	HC Ajoie	05:03	NaN	3
4	4	13	SC Rapperswil-Jona Lakers	SCL Tigers	05:01	NaN	3
5	5	7	ZSC Lions	EHC Biel-Bienne	04:03	NaN	3
6	6	9	EV Zug	SC Bern	03:00	NaN	3
7	7	1	EHC Biel-Bienne	HC Davos	01:04	NaN	0
8	8	5	HC Ajoie	ZSC Lions	00:04	NaN	0
9	9	3	SC Bern	Genève-Servette HC	02:01	SO	2

Features: (364, 2)

	Home_Id	Away_Id
0	0	10
1	1	12
2	2	11
3	3	8
4	4	13

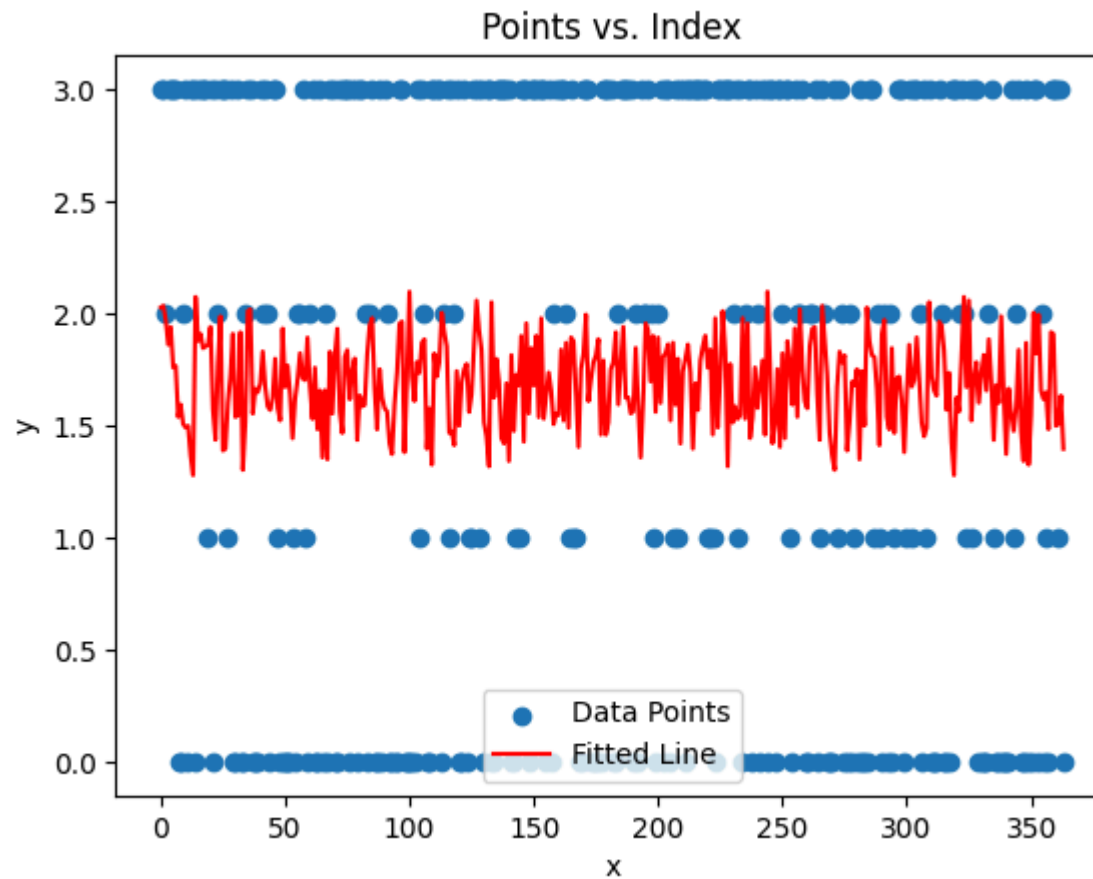
Shape of target variable y: (364,)

```
0    3
1    3
2    2
3    3
4    3
```

Name: Points, dtype: int64

1. Linear Regression

```
In [4]: model=lr.fit(X, y)
        dp.plot_results(df, model)
```



```
In [5]: # train 40 Runden, test Letzte 12 Runden
lr.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=40, test_idx_start=40, test_idx_end=52)
```

Number of training items: 280, test items: 84

Training on rounds 0 to 40, testing on rounds 40 to 52

OLS Regression Results

```

=====
Dep. Variable:          Points    R-squared (uncentered):          0.521
Model:                  OLS       Adj. R-squared (uncentered):        0.518
Method:                 Least Squares    F-statistic:                196.8
Date:                  Sun, 19 Apr 2026    Prob (F-statistic):         1.41e-58
Time:                  18:06:48    Log-Likelihood:             -660.47
No. Observations:      364    AIC:                        1325.
Df Residuals:          362    BIC:                        1333.
Df Model:               2
Covariance Type:       nonrobust
=====

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Home_Id      0.0775      0.014      5.418      0.000      0.049      0.106
Away_Id      0.1405      0.014      9.818      0.000      0.112      0.169
=====

```

```

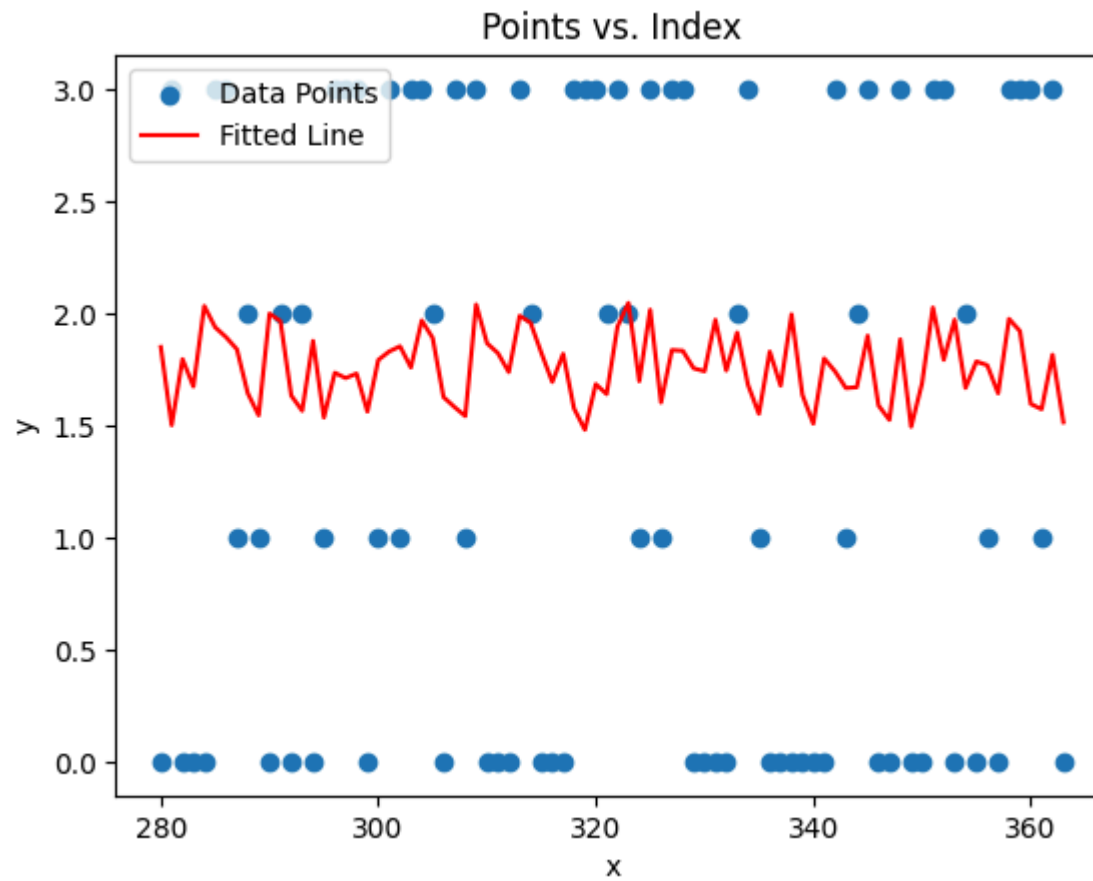
=====
Omnibus:                203.202    Durbin-Watson:                1.837
Prob(Omnibus):           0.000    Jarque-Bera (JB):              23.812
Skew:                    -0.191    Prob(JB):                      6.75e-06
Kurtosis:                1.807    Cond. No.                      2.38
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

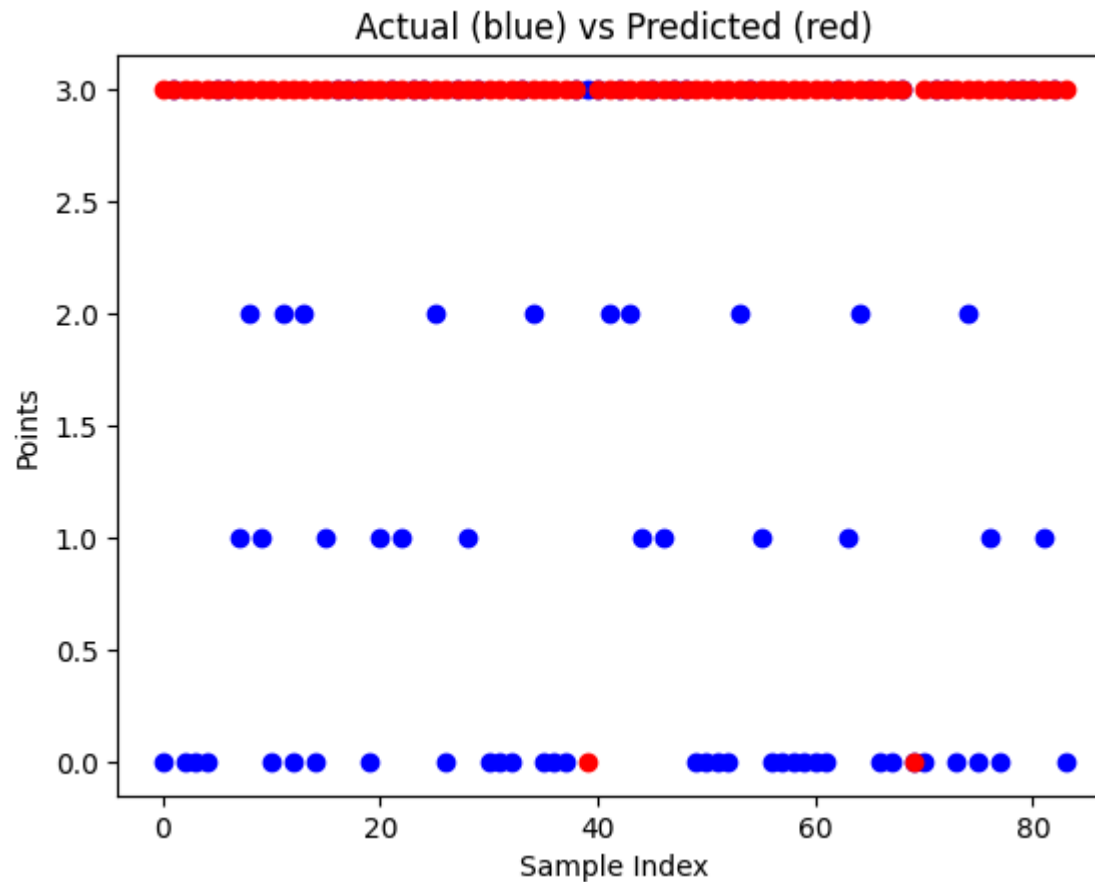


Predicted values $\hat{y}$ are rounded:

Model RMSE: 2.0557700445795186

Model R^2 : -1.4581650317368715

Model Accuracy: 0.34523809523809523



Die vorausgesagten Punkte für sind letzten 12 Runden (84 Spiele) sind rot dargestellt und überschreiben die tatsächlichen Werte (blau). Je mehr man blaue Punkte sieht, desto schlechter ist das Modell. Die Accuracy von 0.35 ist entsprechend schlecht. Raten wäre vermutlich besser.

Aber: Die letzten Runden vor den Playoffs werden ganz anders gespielt und sind oft sehr unvorhersehbar sind, weil Teams, die schon qualifiziert sind, nicht mehr so motiviert sind, und Teams, die sich noch qualifizieren müssen, alles geben. Deshalb ist es vermutlich **falsch auf die ersten Runden zu trainieren und den letzten Runden zu testen**.

```
In [6]: # train 10 Runden, test nächste 10 Runde
lr.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20)
```

Number of training items: 70, test items: 70

Training on rounds 0 to 10, testing on rounds 10 to 20

OLS Regression Results

```

=====
Dep. Variable:          Points    R-squared (uncentered):          0.521
Model:                  OLS       Adj. R-squared (uncentered):        0.518
Method:                 Least Squares    F-statistic:                196.8
Date:                  Sun, 19 Apr 2026    Prob (F-statistic):         1.41e-58
Time:                  18:06:48    Log-Likelihood:             -660.47
No. Observations:      364    AIC:                        1325.
Df Residuals:          362    BIC:                        1333.
Df Model:               2
Covariance Type:       nonrobust
=====

```

```

=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
Home_Id      0.0775     0.014     5.418     0.000     0.049     0.106
Away_Id      0.1405     0.014     9.818     0.000     0.112     0.169
=====

```

```

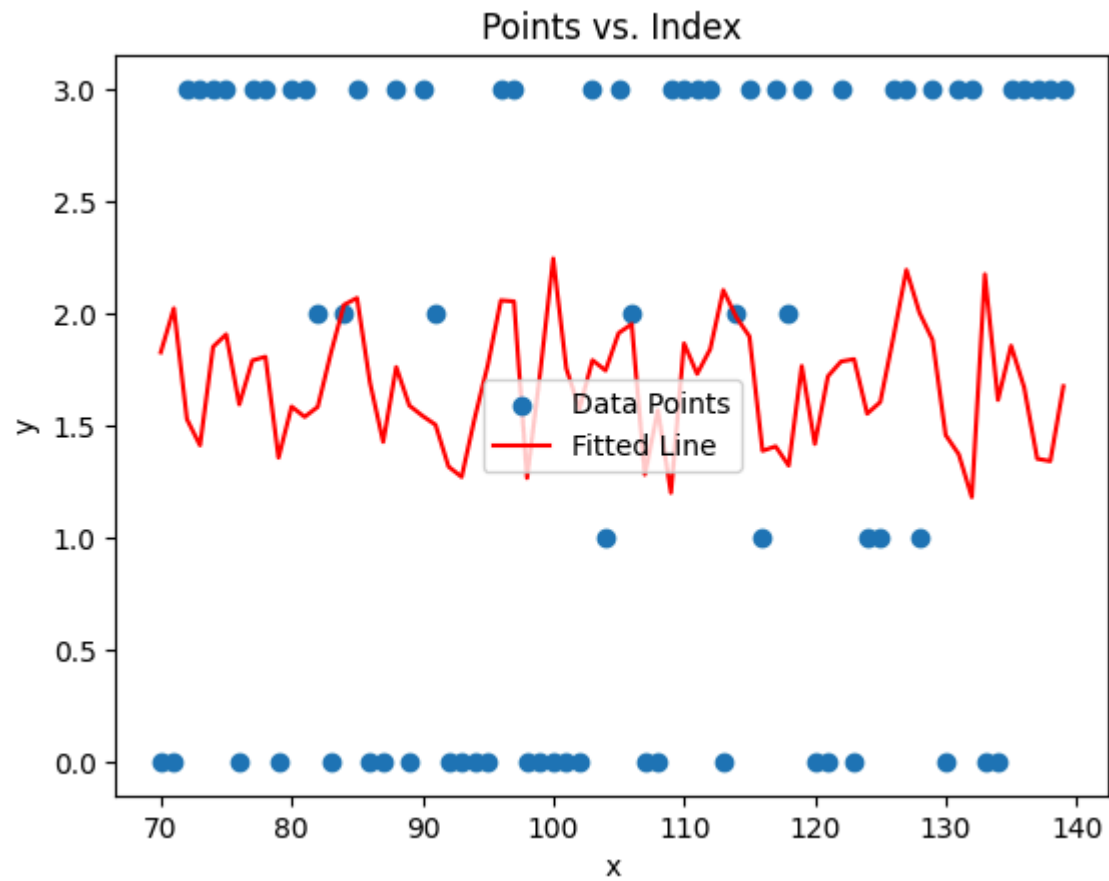
=====
Omnibus:                 203.202    Durbin-Watson:                 1.837
Prob(Omnibus):            0.000    Jarque-Bera (JB):                23.812
Skew:                    -0.191    Prob(JB):                        6.75e-06
Kurtosis:                 1.807    Cond. No.                        2.38
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

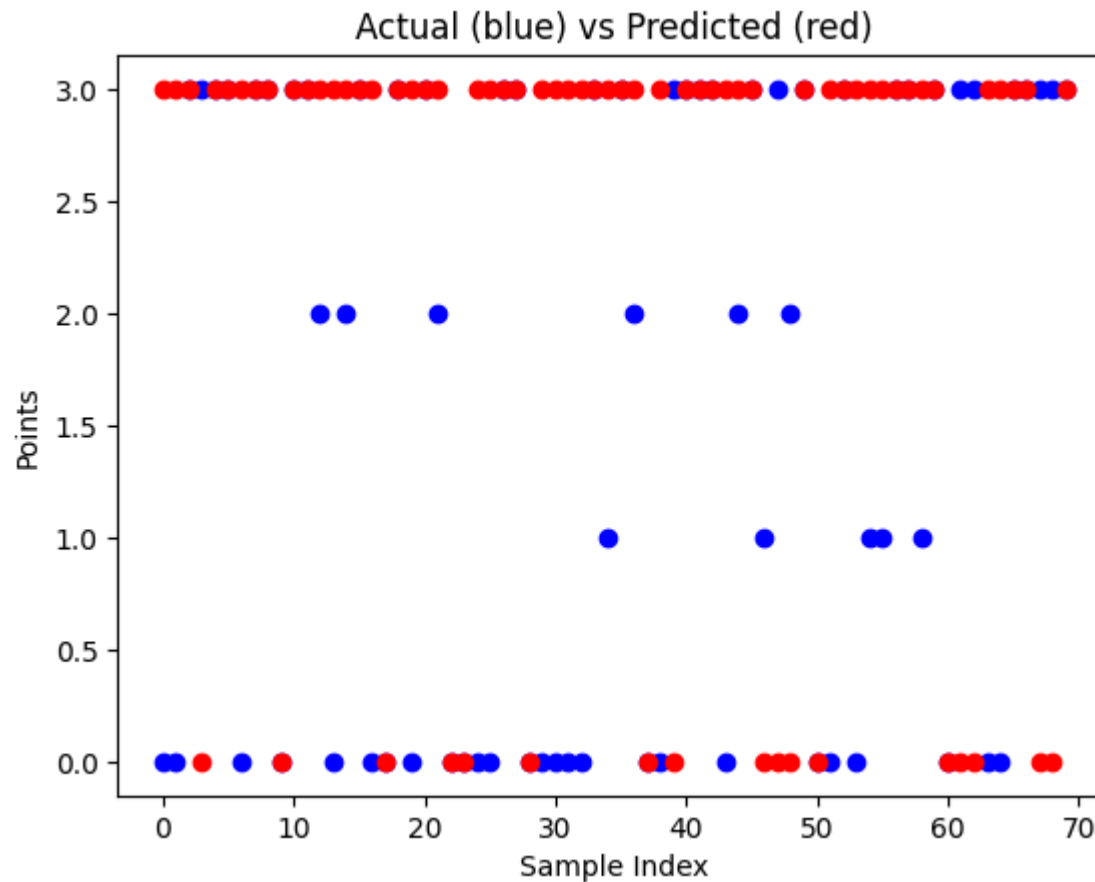


Predicted values $\hat{y}$ are rounded:

Model RMSE: 1.8935982376719422

Model R^2 : -0.8763348996155484

Model Accuracy: 0.4857142857142857



Was sich bestätigt, wenn auf die ersten 10 Runden trainiert und den nächsten 10 Runden getestet, ist die Accuracy (0.49) besser.

2. Voraussage nach Ranking

Vorhersage der Punkte basierend auf der Rangliste der Teams, die aus den vorherigen Spielen erstellt wird. Diese Taktik sollte man schlagen können.

```
In [7]: rr.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20)
```

Number of training items: 70, test items: 70

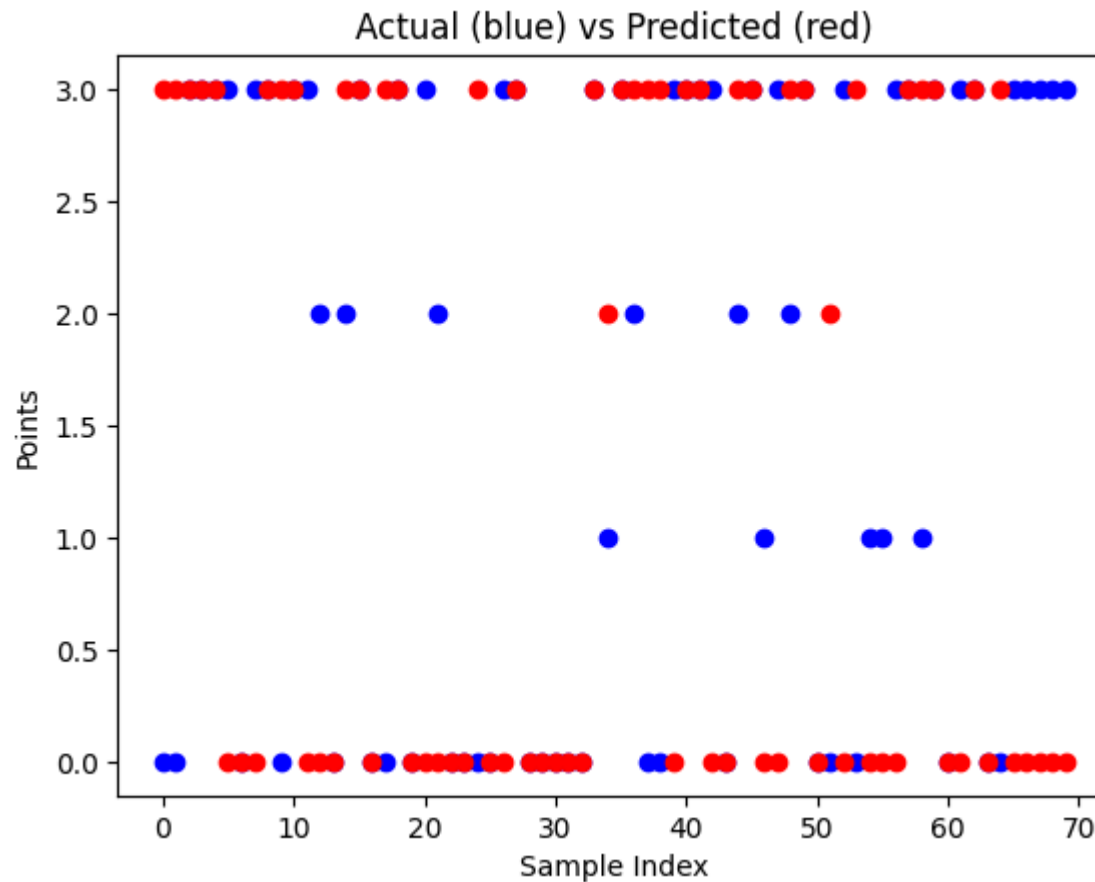
Training on rounds 0 to 10, testing on rounds 10 to 20

	Team	Punkte	Spiele
0	Lausanne HC	27	11
1	HC Davos	26	10
2	Fribourg-Gottéron	21	10
3	Genève-Servette HC	20	10
4	SC Rapperswil-Jona Lakers	20	10
5	ZSC Lions	17	10
6	EV Zug	17	10
7	EHC Kloten	12	11
8	SCL Tigers	11	10
9	EHC Biel-Bienne	11	9
10	HC Lugano	8	10
11	SC Bern	8	9
12	HC Ambri-Piotta	7	10
13	HC Ajoie	5	10

Model RMSE: 1.886038933093073

Model R²: -0.8613840239214008

Model Accuracy: 0.4714285714285714



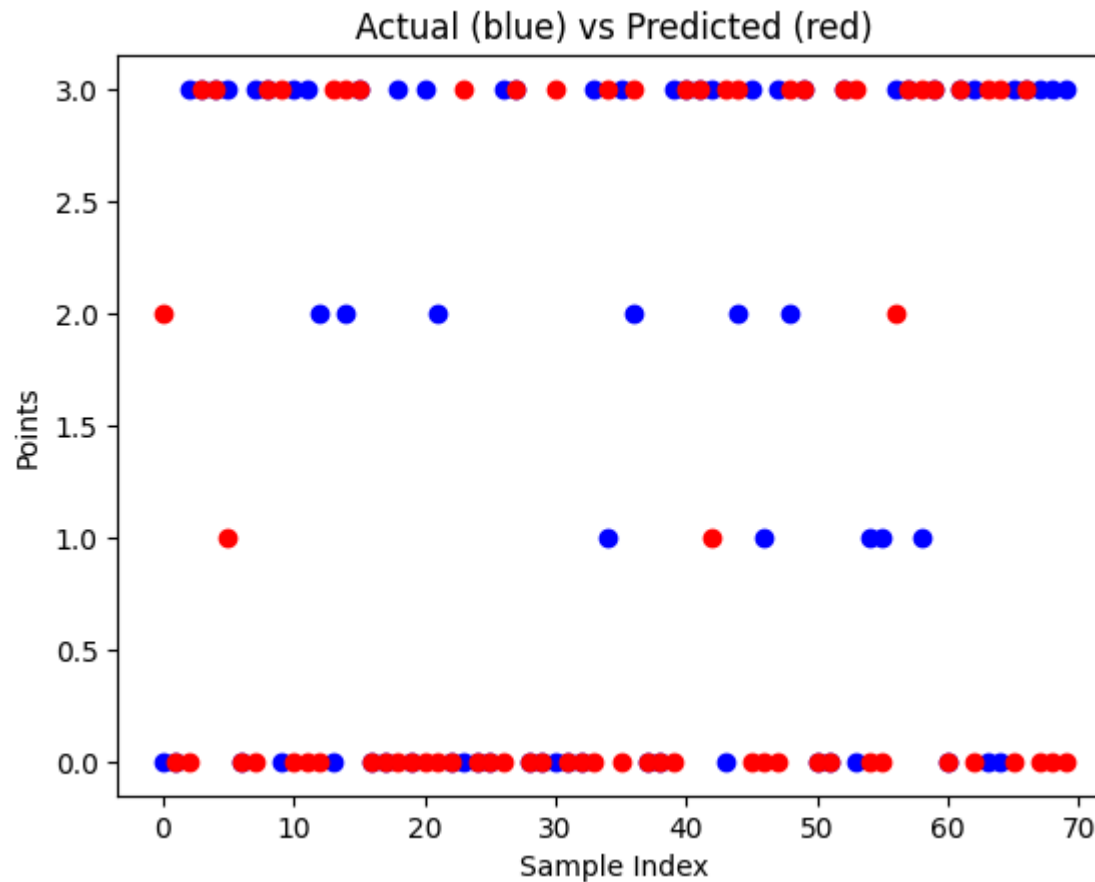
Accuracy ist ca. 0.47, vergleichbar mit linearer Regression

3. KNN: k-Nearest-Neighbor-Algorithmus

Auch mit KNN, und dem Trainieren auf den ersten 10 Runden und dem Testen auf den auf den folgenden 10 Runden, erhalten wir eine ähnlich Accuracy.

```
In [8]: knn.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20)
```

Number of training items: 70, test items: 70
Training on rounds 0 to 10, testing on rounds 10 to 20
Model RMSE: 1.9309509130403675
Model R^2 : -0.9510892780862874
Model Accuracy: 0.42857142857142855

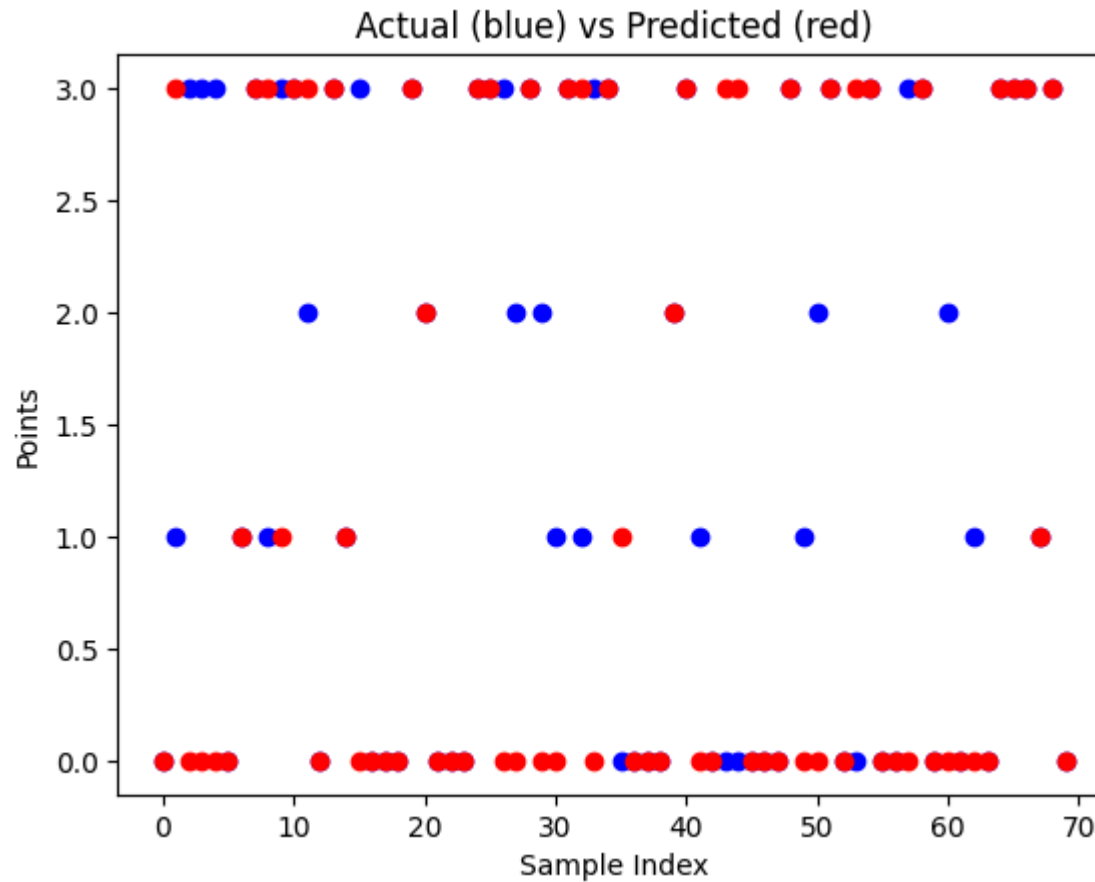


3.1 Ein Beispiel für Overfitting

Trainiert man alle Spiele, erhält man ein viel bessere Accuracy. Deshalb ist so wichtig Train-Set und Test-Set unterschiedlich zu halten um eine richtige Crossvalidierung zu erhalten.

```
In [9]: knn.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=52, test_idx_start=42, test_idx_end=52)
```

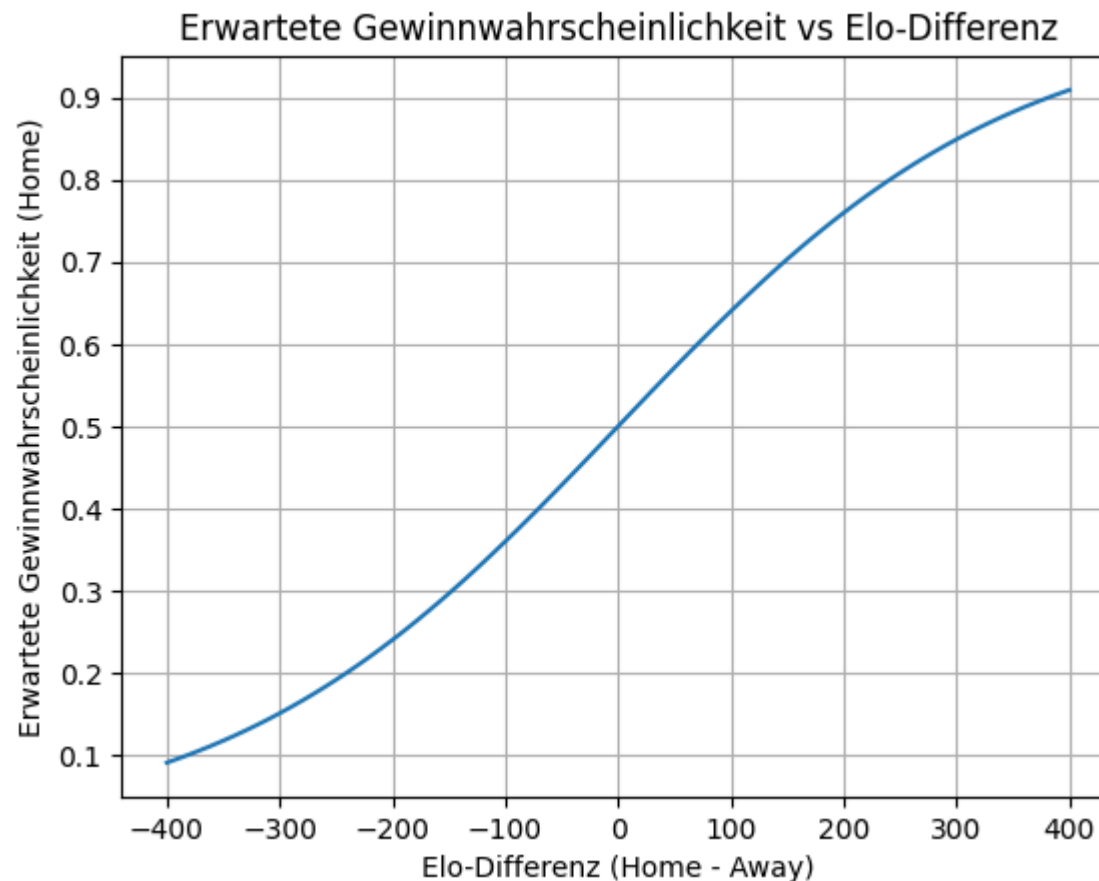
Number of training items: 364, test items: 70
Training on rounds 0 to 52, testing on rounds 42 to 52
Model RMSE: 1.3522468075656264
Model R²: -0.03751736915238557
Model Accuracy: 0.6571428571428571



4. ELO-Rating

ELO Rating kommt eigentlich vom Schach und dient dazu die Spielstärke eines Spielers zu bestimmen, ohne dass alle Spieler gegeneinander gespielt haben müssen. Das kann man auch für Teams anwenden, indem man z.B. die letzten 5 Spiele anschaut und die Rangliste der letzten 5 Spiele bestimmt. Mit Elo ist das aussagekräftiger als nur eine Rangliste von Punkten, weil das Gewinnen gegen einen schwachen weniger während gegen einen starken Gegner mehr ELO-Punkte gibt und so eine bessere Aussage liefert.

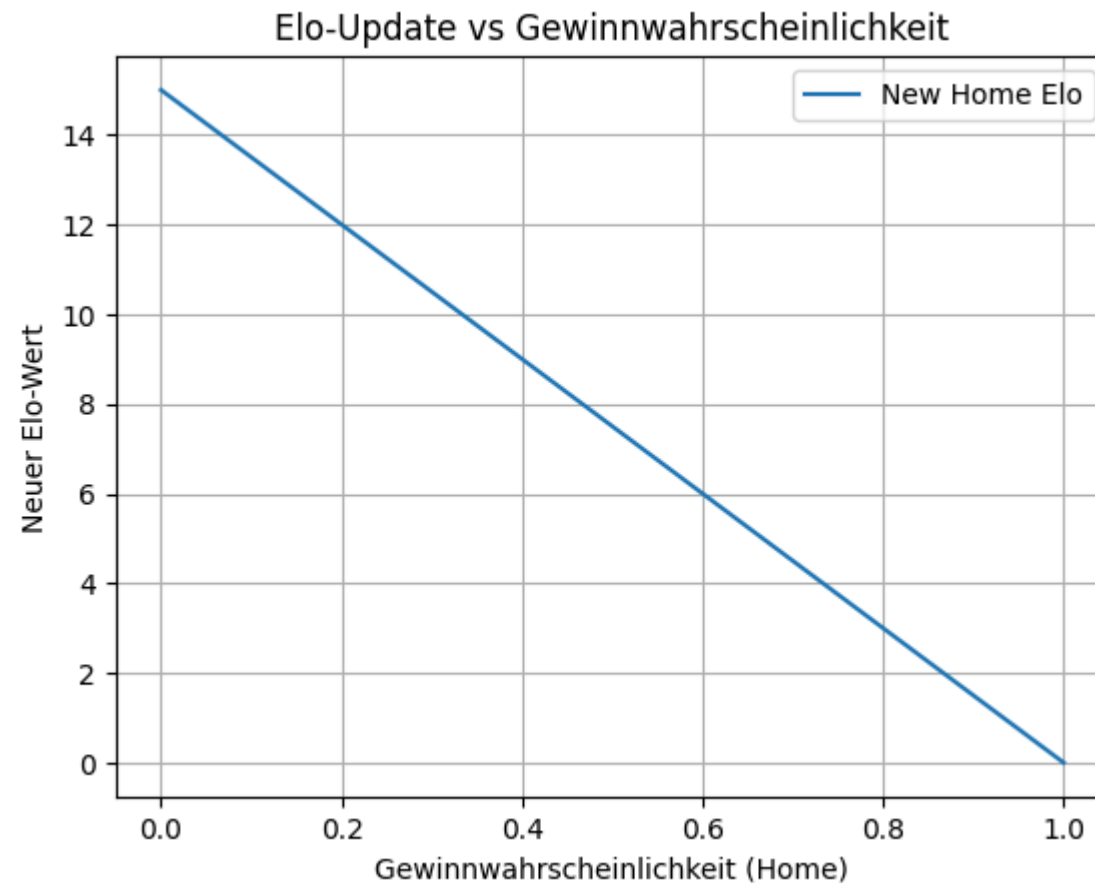
```
In [10]: elo.plot_expected_score()
```



Je nach Elo Differenz berechnet man die Wahrscheinlichkeit zu gewinnen, damit man danach Elo-Punkte berechnen kann, die im Fall eines Sieges dazukommen.

```
In [11]: elo.plot_update_elo()
```

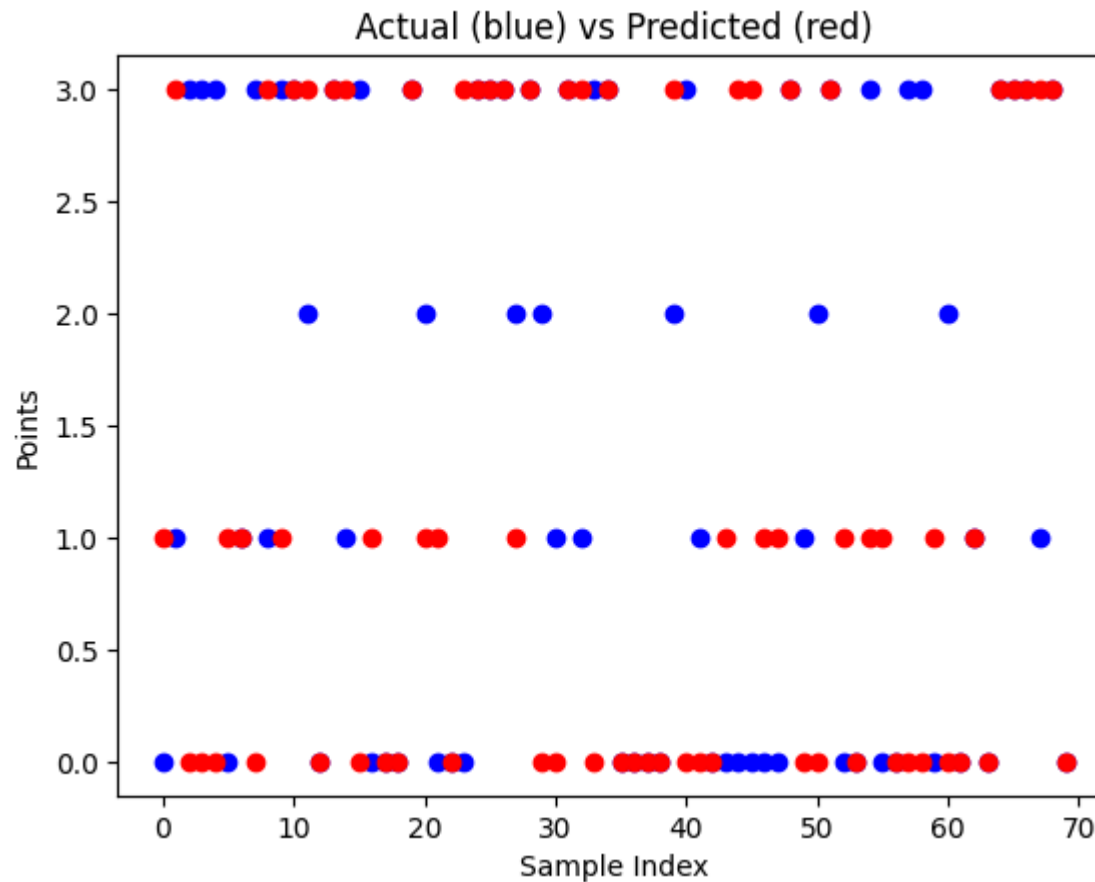
```
score: [0.          0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
0.66666667 0.77777778 0.88888889 1.          ]
score: [0.          0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
0.66666667 0.77777778 0.88888889 1.          ], new_home: [15.0, 13.333333333333332, 11.666666666666666, 10.000000000000002, 8.333333333333334, 6.666666666666666, 5.000000000000001, 3.333333333333335, 1.6666666666666674, 0.0]
```



Damit gewinnt ein Team 15 Punkte falls es kein Chance hat zu gewinnen und 0 falls es immer gewinnt. $K = 15$ ist damit ein Hyperparameter.

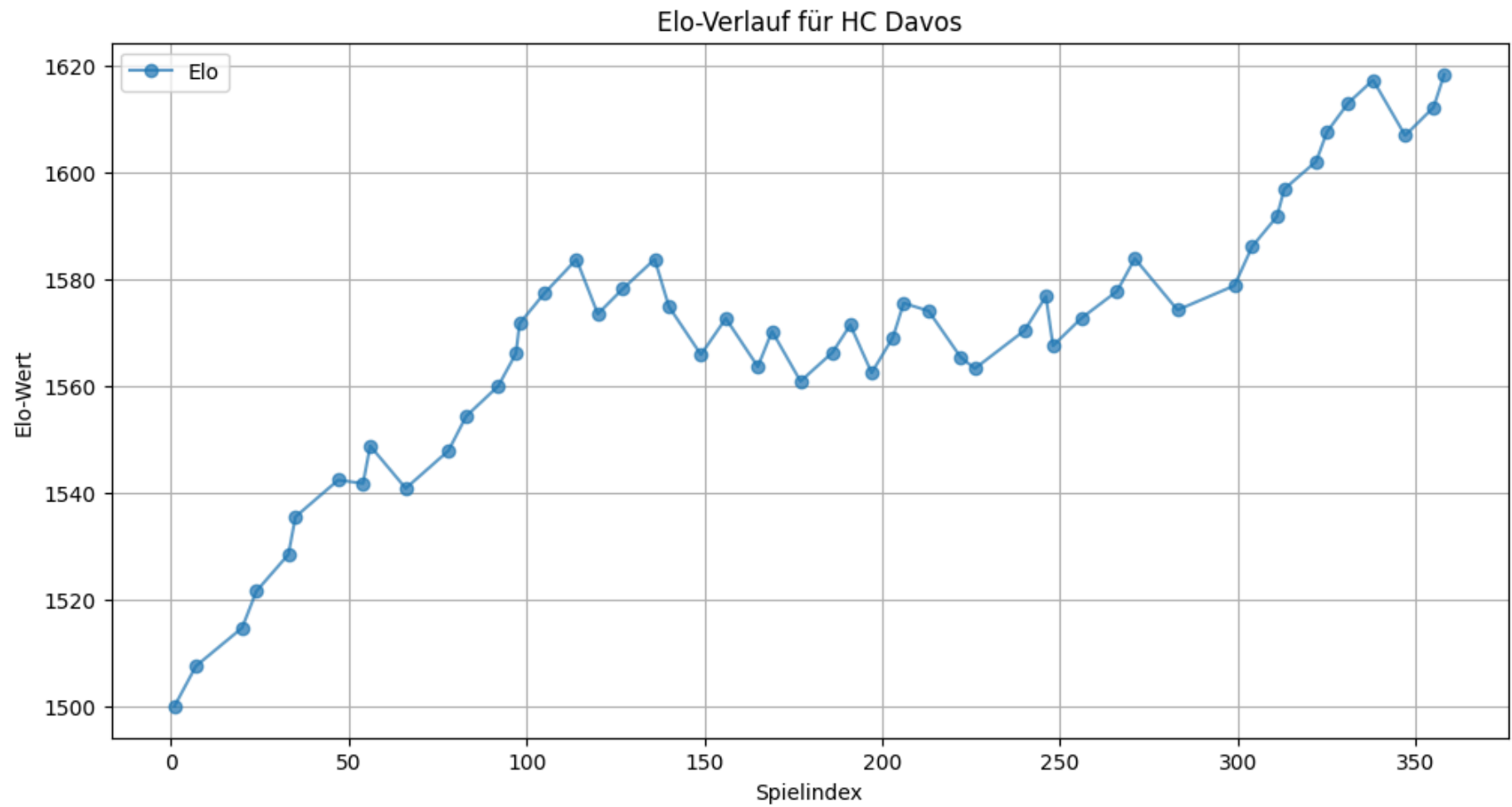
```
In [12]: elo.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=42, test_idx_start=42, test_idx_end=52)
```

Number of training items: 294, test items: 70
Training on rounds 0 to 42, testing on rounds 42 to 52
Model RMSE: 1.535298947157477
Model R^2 : -0.3374247336729972
Model Accuracy: 0.44285714285714284

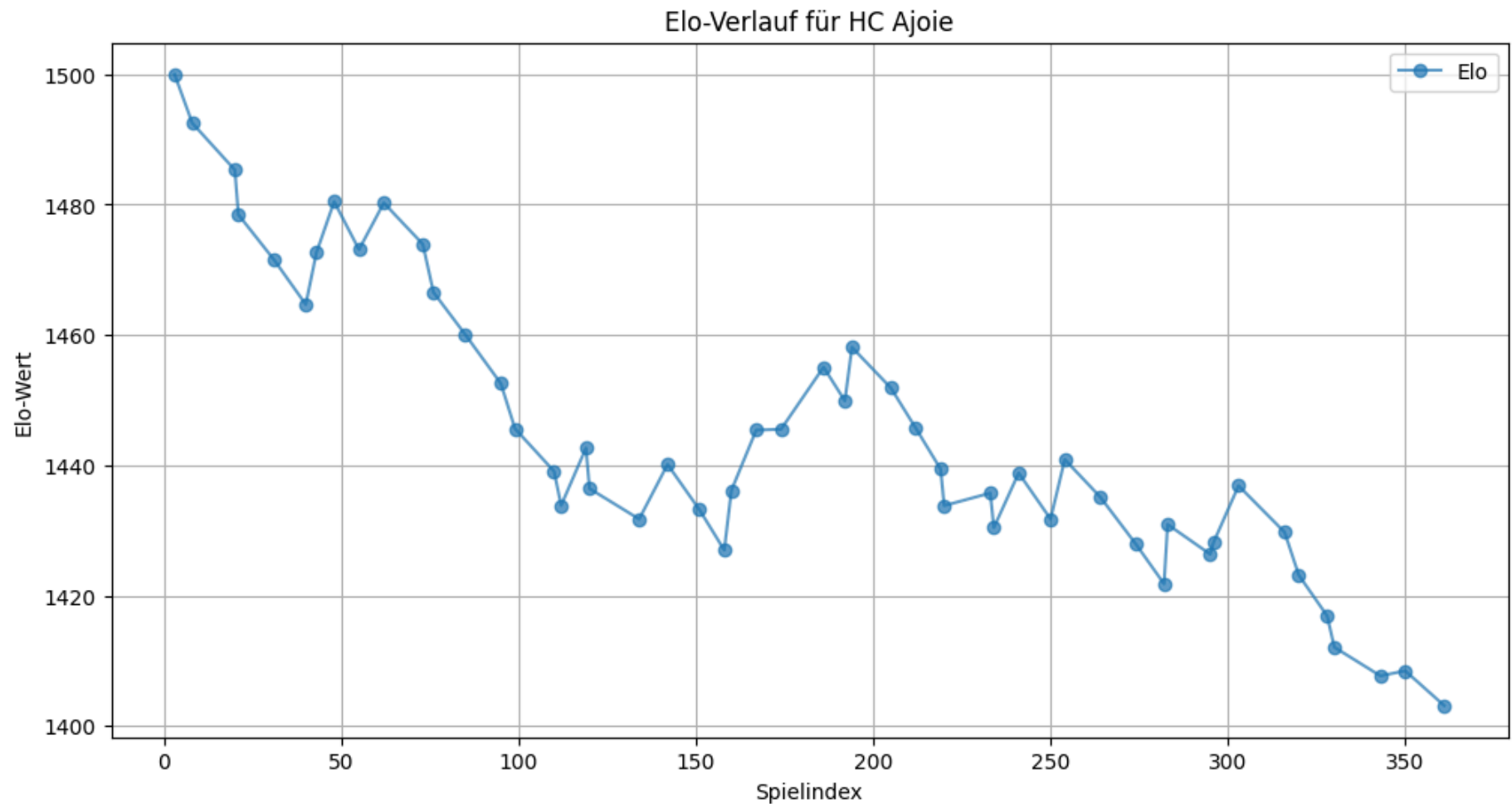


Auch mit Elo erhalten wir eine ähnliche Accuray wie mit Voraussage nach Rangliste. Und hier der Elo-Verlauf vom HC Davos und Ajoie zur Veranschaulichung.

```
In [13]: elo.plot_Elo_HC_Davos_Ajoie(df)
```

Final Elo for HC Davos: 1618.5



Final Elo for HC Ajoie: 1403.1

5. Mit Goalie Statistik

<https://www.sihf.ch/de/game-center/national-league/#/goalkeepers/goalsAgainstAverage/asc/page/0/2026/4940> --> alle Spiele

<https://www.sihf.ch/de/game-center/game/#/20261105000001> -> ein Spiel

Seite laden, danach auf Player Stats --> Goalie Stat SVS%

Hier kann man pro Spiel die Goalie Statistik herausholen

Mit nur dem Spielresultat lässt sich praktisch keine gute Voraussage machen, ca. 45% Treffer (Accuracy). Nimmt man aber andere Features die das Spiel beeinflussen wie die Goalie-fitness. Kann man die Voraussage deutlich verbessern, sofern man dann die Form des Torwarts in Vorraus weiss. Am besten man verwendet die Torstatistik SVS% (Anzahl gehaltener Schüsse im Verhältniss zu Anzahl Schüsse aufs Tor.

5.1 Lineare Regression

```
In [14]: X, y, df = dp.load(SVS=True)
print(f"Features: {X.shape}")
X.info()
```

```
Features: (364, 4)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 364 entries, 0 to 363
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Home_Id     364 non-null    int64
1   Away_Id     364 non-null    int64
2   SVS_Home    364 non-null    float64
3   SVS_Away    364 non-null    float64
dtypes: float64(2), int64(2)
memory usage: 11.5 KB
```

```
In [15]: lr.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20, SVS=Tru
```

Number of training items: 70, test items: 70

Training on rounds 0 to 10, testing on rounds 10 to 20

OLS Regression Results

```

=====
Dep. Variable:          Points    R-squared (uncentered):          0.715
Model:                  OLS      Adj. R-squared (uncentered):        0.712
Method:                 Least Squares    F-statistic:                226.1
Date:                  Sun, 19 Apr 2026    Prob (F-statistic):        7.93e-97
Time:                  18:06:49    Log-Likelihood:            -565.75
No. Observations:      364    AIC:                        1140.
Df Residuals:          360    BIC:                        1155.
Df Model:              4
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Home_Id	-0.0360	0.015	-2.399	0.017	-0.066	-0.006
Away_Id	0.0218	0.015	1.451	0.148	-0.008	0.051
SVS_Home	0.1108	0.009	11.690	0.000	0.092	0.129
SVS_Away	-0.0908	0.009	-9.607	0.000	-0.109	-0.072

```

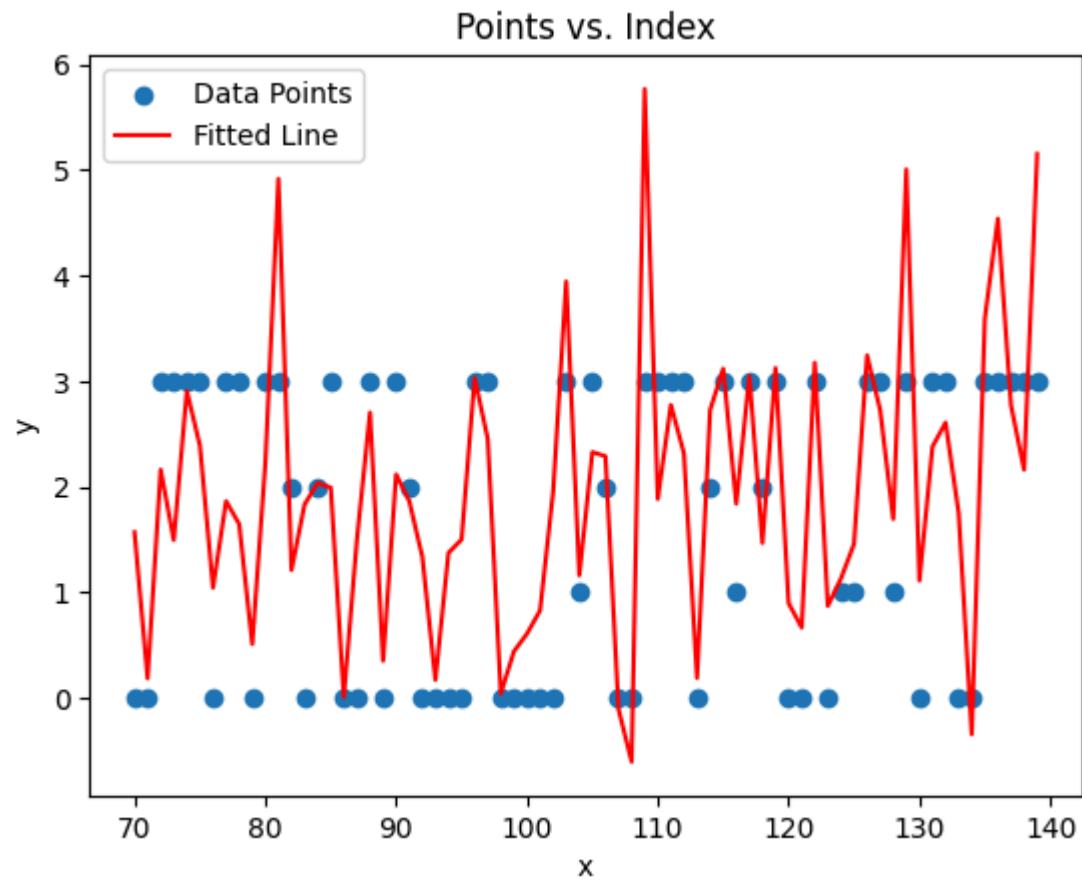
=====
Omnibus:                1188.046    Durbin-Watson:              1.804
Prob(Omnibus):           0.000    Jarque-Bera (JB):           29.586
Skew:                    -0.137    Prob(JB):                   3.76e-07
Kurtosis:                1.631    Cond. No.                   33.3
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

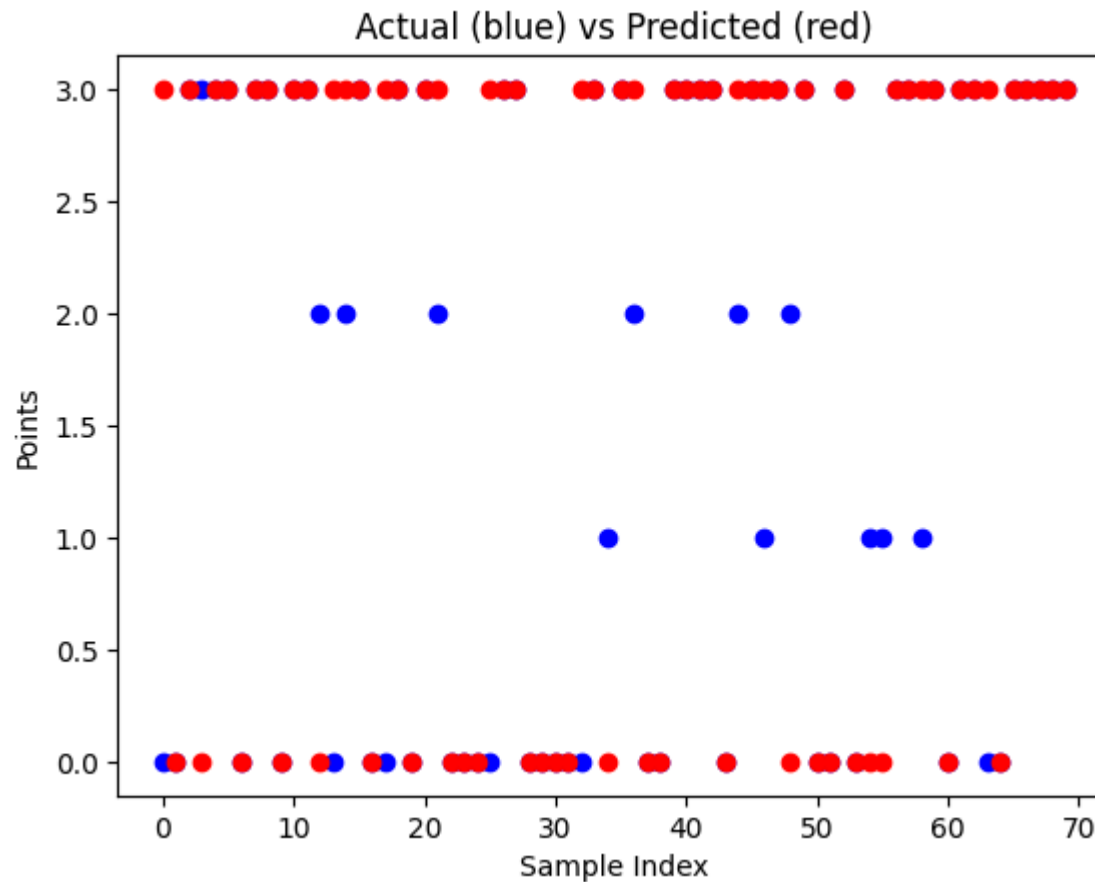


Predicted values $\hat{y}$ are rounded:

Model RMSE: 1.1084094137869043

Model R^2 : 0.3571123451516447

Model Accuracy: 0.7428571428571429

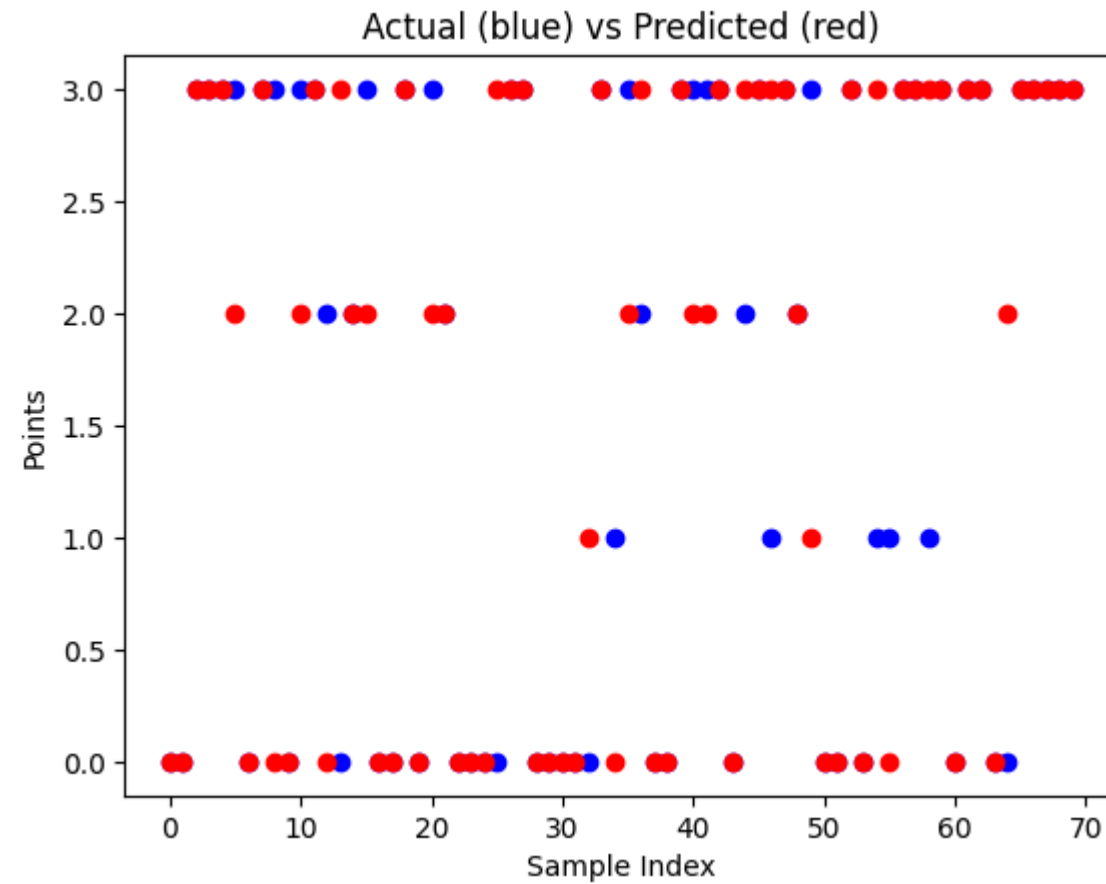


Bemerkenswert ist, dass die SVS% sehr signifikanten Einfluss ($p\text{-value} < 0.05$) hat, während die Away-Id eigentlich keinen signifikanten Einfluss hat. Die Accuracy von 74% ist natürlich traumhaft, wenn man die Fitness des Torwarts voraussagen kann. In dieser Genauigkeit wie aus den Testdaten entnommen wurde, die tatsächlichen Ergebnisswerte, wird man das nie können. Deshalb ist das eigentlich nur eine Aussage, die jedermann im Elshockey schon kennt, dass der Torhüter mehr als 50% ausmacht.

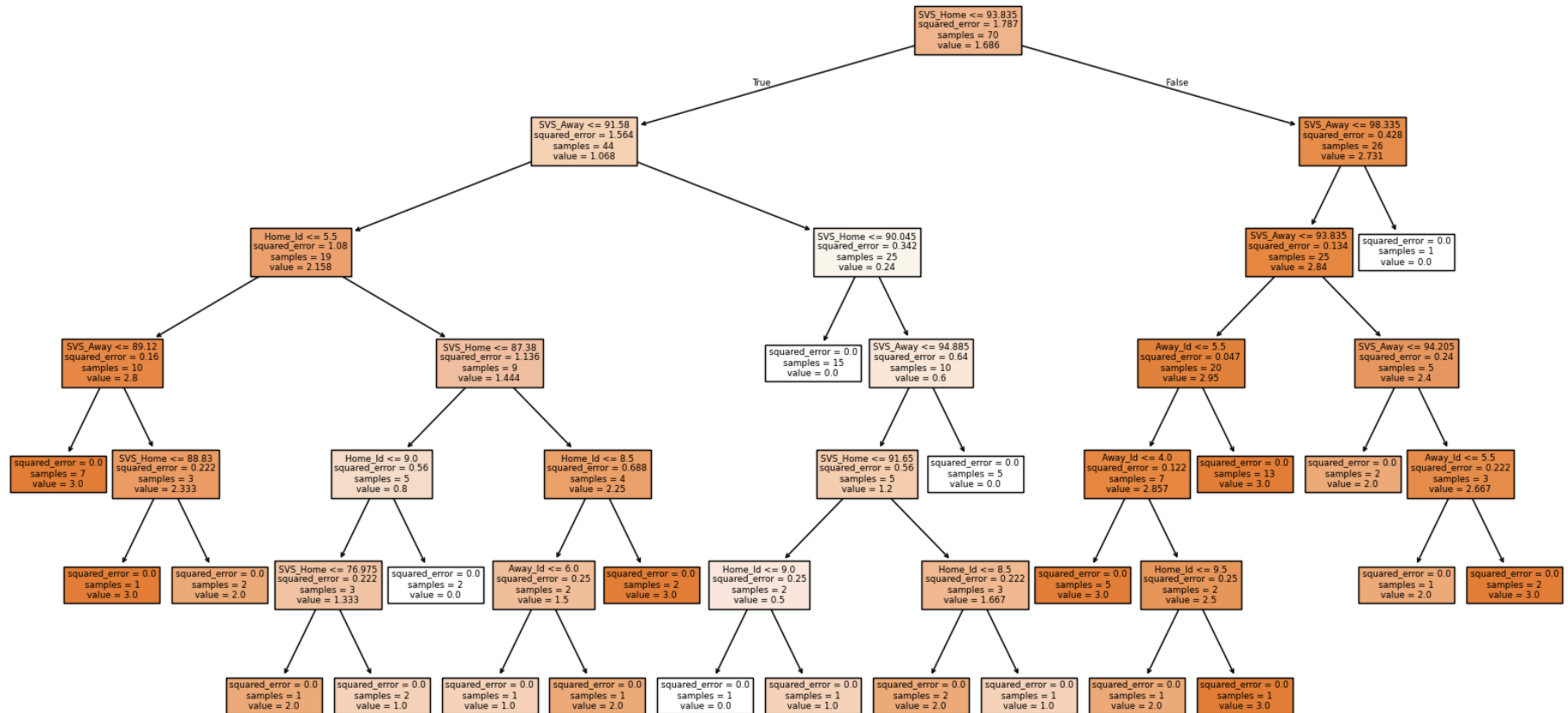
5.2 Entscheidungsbaum

```
In [16]: rp.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20)
```

Number of training items: 70, test items: 70
Training on rounds 0 to 10, testing on rounds 10 to 20
Model RMSE: 0.9486832980505138
Model R^2 : 0.5290474156343443
Model Accuracy: 0.7



Decision Tree



5.3 Random Forest Classifier

```
In [17]: rf.analyze_model_performance(df=df, X=X, y=y, train_idx_start=0, train_idx_end=10, test_idx_start=10, test_idx_end=20)
```

Number of training items: 70, test items: 70

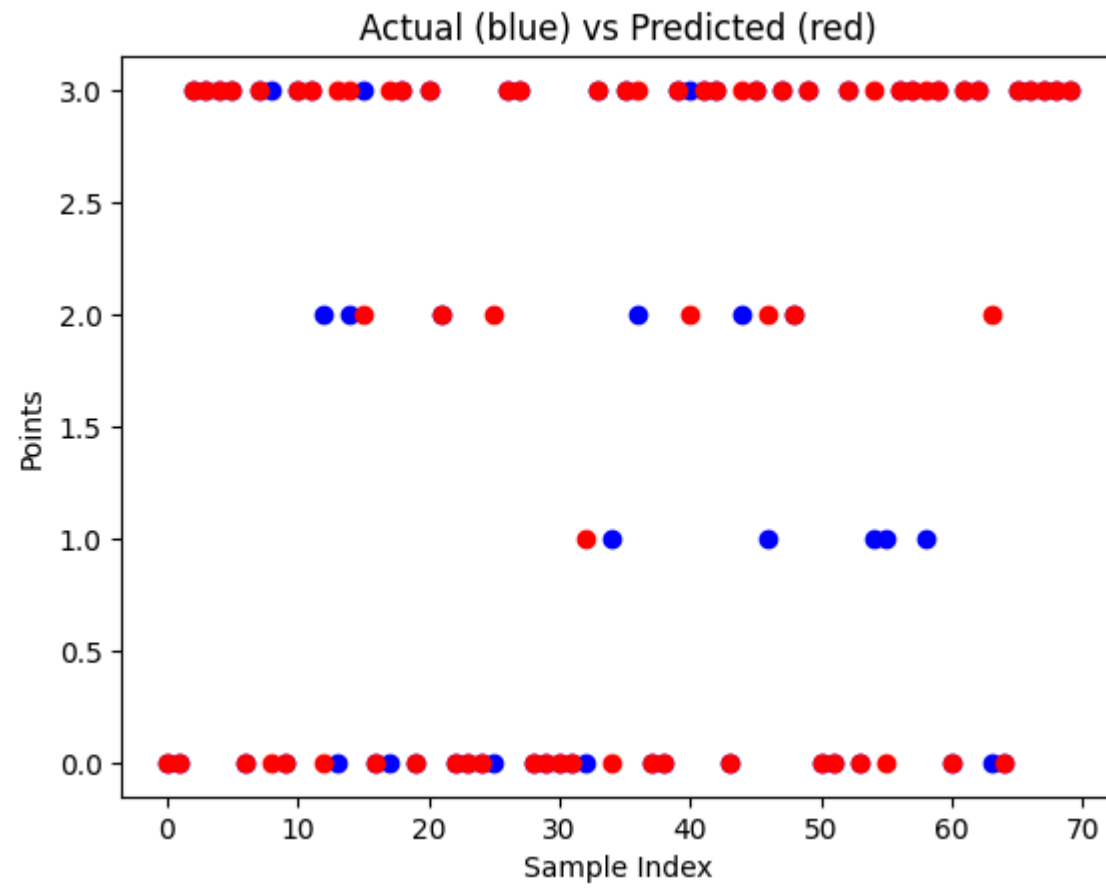
Training on rounds 0 to 10, testing on rounds 10 to 20

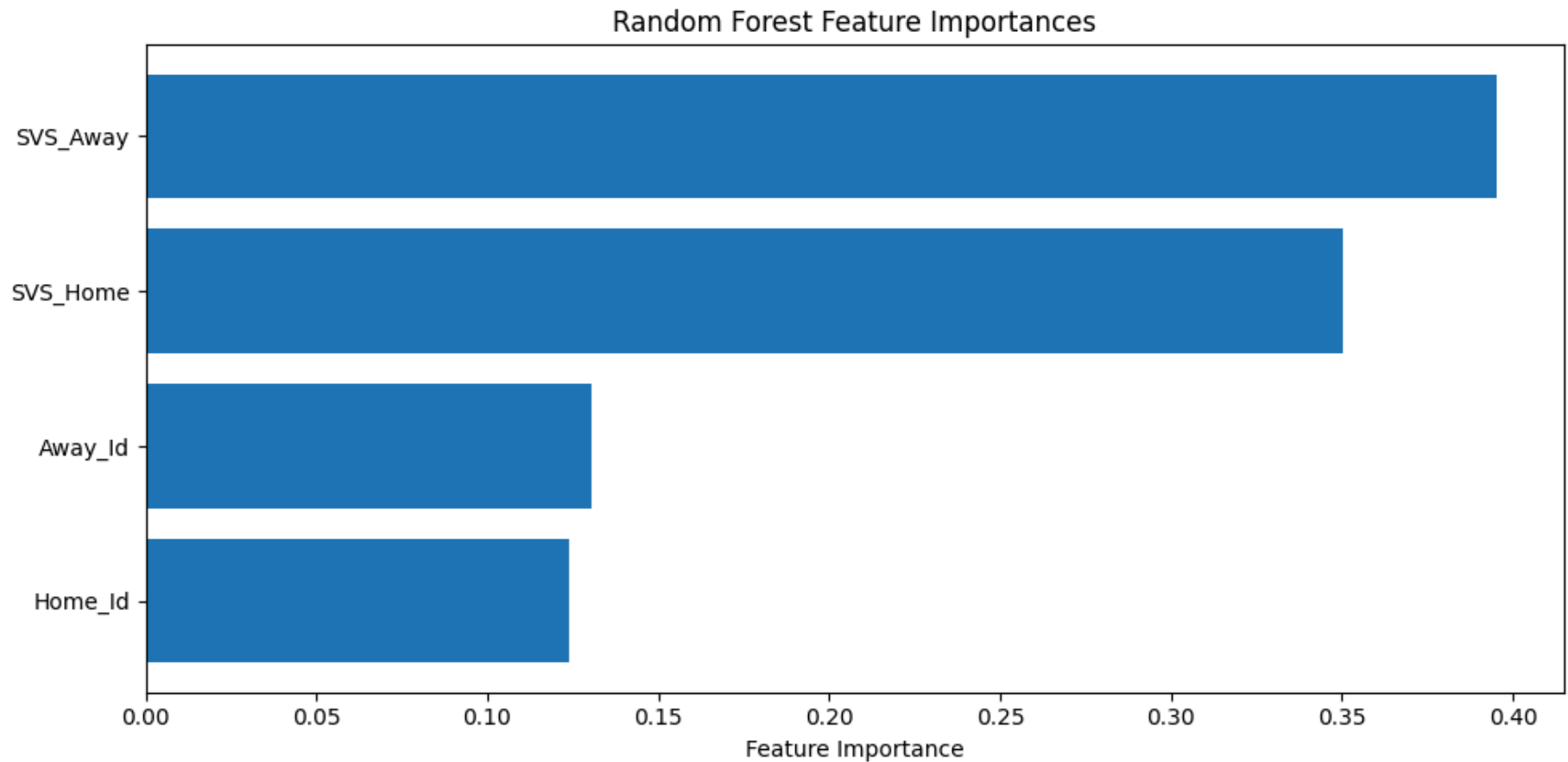
y_hat: [0 0 3 3 3 3 0 3 0 0]

Model RMSE: 0.8944271909999159

Model R²: 0.5813754805638617

Model Accuracy: 0.7571428571428571





6. Zusammenfassung

Zusammenfassend ist Eishockey eine sehr volatile Sportart. Aus den Resultaten alleine lässt sich kaum eine verlässliche Voraussage machen. Weiss man mehr über die Spieler, ist der Topscorer verletzt oder der Torhüter angeschlagen, lässt sich die Voraussage deutlich verbessern. Aber wie weiss man das.

In []: